Biweekly Assessment BW10 — Weeks 19–20

Sessions covered: S91–100
Topics: Testing/debugging/clean code, AI integration lite, resume/interview/placement mocks
Timebox: 45–60 minutes (homework or Friday of the even week).
Rules: Closed notes unless your trainer allows open notes. No copying.
Coding constraint: Testing + career readiness.

A. Multiple Choice (12)

Circle the best option.

1. `unittest` assertions check:
   a) Expected vs actual behaviour
   b) CSS colors
   c) Git blame
   d) SQL charset only
2. A good unit test is:
   a) Needs production DB always
   b) Manual only
   c) One giant test for whole app only
   d) Fast, focused, repeatable
3. Clean code prefers:
   a) Copy-paste everywhere
   b) No comments when complex forever
   c) Clear names + small functions
   d) One 2000-line function
4. Debugging first step often is:
   a) Disable HTTPS
   b) Reproduce + read the error
   c) Rewrite entire app
   d) Delete git history
5. AI coding assistants are best used to:
   a) Accelerate with your review
   b) Blindly ship secrets
   c) Skip learning basics
   d) Replace code review forever
6. Resume bullet for projects should show:
   a) Only course name
   b) Only soft skills adjectives
   c) Passwords
   d) Impact + tech + your role
7. STAR method is for:
   a) CSS grid
   b) React keys
   c) Interview storytelling
   d) SQL joins

8. `assertRaises` tests:
   a) That npm builds
   b) That code raises an expected exception
   c) That CSS loads
   d) That git pushes
9. Technical interview prep should include:
   a) Explaining your project end-to-end
   b) Memorizing only buzzwords
   c) Avoiding GitHub
   d) Hiding bugs always
10. Logging in tests/prod helps:
    a) Replace unit tests completely
    b) Style pages
    c) Create React apps
    d) Diagnose failures
11. A portfolio README should include:
    a) API keys
    b) Unrelated memes only
    c) Setup steps + screenshots/demo
    d) Only your photo
12. Mock interviews help because:
    a) Avoid speaking English
    b) Practice under pressure + feedback
    c) Replace building projects
    d) Skip fundamentals

B. Coding (2)

1. Write two `unittest` methods for a function `average(marks)`: one normal list, one empty list expecting `ValueError`.

```
# your code
```

2. Write a 4-bullet project description for your full-stack capstone (problem, stack, your features, result).

```
# your code
```

C. Short answer (1)

Name three things you will demo in a placement mock for your capstone.

(Write 3–6 sentences.)